Thông tin đề thi

Mã đề: PE_SDN302_SPRING 2026 **Course**: SDN302 **Duration**: 85 minutes **Tools**: Visual Studio Code, MongoDBCompass, Postman, MongoDB V6 or later

INSTRUCTIONS

Read the instructions carefully before starting the questions.

- You are **NOT allowed** to use any device to share data with others.
- **Visual Studio Code**, **MongoDBCompass**, **Postman**, and **MongoDB V6 or later** must be used as your development tools.
- StudentCode: e.g., **se181234**
- Yourdomain: hostname+port; e.g., localhost:5000

IMPORTANT – the following steps MUST be done before you start your solution:

1. Before submitting your work, check that there are no syntax error(s) in the submitted PE.
2. The data should be obtained from the provided JSON files.
3. **If there are syntax errors or compilation errors in your PE program, you will not pass the PE requirements, the mark will be 0.**
4. In Visual Studio Code, create a new project named **SDN302_SP26_StudentCode**.
5. Create a Mongo database named **SDN302_SP26_StudentCodeDB**.
6. **Your submission will be invalid (0 points) if it contains irrelevant content. Store your MongoDB connection string, port value, and JWT secret key in the .env file. Failure to do so will result in 0 points.**

Mô tả bài toán

Build a part of the **Pen Management System**. The scenario is as follows: A brand has many pens, and each pen belongs to one brand.

Your database consists of **3 collections** with the following schema:

```
brands ({
    brandName: {type: String, required: true, unique: true},
    ballpoint: {type: Boolean, default: false},
}, {timestamps: true});

pens ({
    penName: {type: String, required: true, unique: true},
    penDescription: {type: String, required: true},
    image: {type: String, required: true},
    pouch: {type: Boolean, default: false},
    off: {type: Number, required: true},
    gender: {type: Boolean, default: false},
    brand: {type: mongoose.Schema.Types.ObjectId, ref: "brand", required: true},
}, { timestamps: true, });

members ({
    name: {type: String, required: true},
    code: {type: String, required: true},
}, {timestamps: true});
```

Task 01 (1.0 mark)

Use the Express generator tool to develop the new REST API server, create the related brands, and migrate them to a database named **SDN302_SP26_StudentCodeDB** (MongoDB).

Task 02 (5.0 marks)

Create a new REST API server for the **brand** collection. You must use **JSON Web Token (JWT)** for authenticating the API. The secret key of JWT must be **your StudentCode! (e.g., se181234!) and store in the .env file**. The route paths will match requests to:

- Visit http://yourdomain/auth/tokens to generate the JWT. The account credentials are stored in the members.json file (name: admin, code: 123456789). **Note:** Do not store the JWT in cookies or session storage.
- http://yourdomain/rest/brands and http://yourdomain/rest/brands/:id for all methods related to brands.
- Implement JWT-protected CRUD operations for the brand collection.
- For the DELETE operation, ensure a brand cannot be deleted if any pens are associated with it (i.e., if there are documents in the pen collection referencing the brand's _id). Return an appropriate error message (e.g., "Cannot delete brand because it has associated pens") if deletion is attempted in such cases.

*— Tài liệu được thành viên đăng tải tại **FUOVERFLOW** và được biên soạn lại để hỗ trợ ôn tập —*

Task 03 (4.0 marks)

Client Application for Pen Management

Students must select one of the template engines (EJS, HANDLEBARS, or PUG) with Express for building the views.

1. **Authentication**

   At the login view, the route path will match requests to: **http://yourdomain/auth/access**. Users must enter their account name (*name*) and code (*code*) correctly.

   - The code must be encrypted using bcrypt and stored in the members collection.
   - All CRUD actions require authentication.
   - Show error messages when login attempts fail. Redirect to the pen list after a successful login.
   - Import account data from members.json into the members collection.

2. **Display Pen List**

   After successful login:

   - Display all pens in a card view (not a table view), including: penName, off (displayed as a percentage). E.g., { "off": 0.75 } → 75%), image, gender (displayed as a male/female icon), and brand name.
   - Route path: **http://yourdomain/admin/pens**.

3. **Delete Pen**

   After successful login:

   - Allow deletion of a selected pen with confirmation.
   - Show a notification message indicating the result.
   - Route path: **http://yourdomain/admin/pens/:id**

4. **Add New Pen**

   After successful login, add a new pen that satisfies the requirements using a modal form.

   - All fields are required.
   - *penName* must contain only letters (a-z, A-Z) and space, and must be unique.
   - *pouch* must be a toggle or switch control.
   - *off* must be entered as a decimal number between 0 and 1 and displayed as a corresponding percentage. E.g., { "off": 0.75 } → 75%
   - *gender* must be a toggle or switch control.
   - *brand* must be a select control displaying brandName values from the brands collection.
   - Route path: **http://yourdomain/admin/pens**.

5. **Update Pen**

   After successful login:

   - Allow updating the pen information.
   - All controls must satisfy the same requirements as the add function.
   - Route path: **http://yourdomain/admin/pens/:id**.

-The end of test-